

Contexto

Na Fase anterior o Sonora ganhou a hierarquia Conteúdo, com Musica e Podcast herdando dela. Funcionou, mas tem um detalhe estranho: hoje nada impede alguém de escrever `new Conteúdo(...)` e dar play num "conteúdo genérico", que não é música, nem podcast, nem nada. No mundo real esse objeto não existe.

Ao mesmo tempo, o Sonora vai ganhar planos de assinatura, do jeito que você já conhece dos apps de streaming: o gratuito com anúncios, o individual e o família. O problema é que um estagiário já começou esse código, e fez do jeito mais direto possível: três classes independentes, cheias de coisa repetida. Seu trabalho nesta fase é arrumar as duas situações com as ferramentas da segunda parte de herança: generalização, classes e métodos abstratos, e a palavra `final`.

Preparação

Como nas fases anteriores, esta atividade é incremental. Clone o projeto da fase passada e trabalhe por cima dele:

- Copie sonora da lista 08 para uma nova pasta sonora-lista09.
- Mantenha o pacote sonora e tudo que já funcionava.
- Nada do que você já entregou pode quebrar: ao terminar essa fase, o projeto continua compilando e rodando.

O que você vai praticar

- Generalização: encontrar os membros comuns de um conjunto de classes e subi-los para uma superclasse.
- Generalização aplicada mais de uma vez, em um conjunto menor de classes.
- Classe abstrata: uma superclasse que representa um conceito genérico e não pode ser instanciada.
- Método abstrato: obrigar as subclasses a implementar um método que não tem implementação na superclasse.
- `final` em método (impedir a sobrescrita) e em classe (impedir a extensão).

Parte A: o diagrama vem antes do código

Generalização é, antes de tudo, uma decisão de projeto. Por isso, antes de escrever qualquer código, leia as Partes B a E inteiras e atualize o diagrama de classes do Sonora com o resultado que você pretende construir:

1. A hierarquia dos planos depois da generalização, com as setas de herança apontando para as superclasses e cada membro na classe onde ele deve morar.
2. As classes abstratas e os métodos abstratos com o nome em itálico, como manda a UML.
3. Conteúdo atualizada (agora abstrata) e a associação entre `Usuario` e `Plano` com os quatro adornos que você já pratica: papel, nome, multiplicidade e navegabilidade.

Use esse diagrama como plano de implementação. Se, durante o código, você perceber que alguma decisão estava errada, corrija o diagrama também: ele precisa terminar coerente com o que foi implementado.

Parte B: generalize os planos do estagiário

Este é o código que o estagiário deixou. Copie as três classes para o seu projeto e compile antes de mexer, para ter certeza de que o ponto de partida funciona.

```
public class PlanoGratuito {

    private String nome;
    private int maxDispositivos;

    public PlanoGratuito() {
        this.nome = "Gratuito";
        this.maxDispositivos = 1;
    }

    public String getNome() { return nome; }
    public int getMaxDispositivos() { return maxDispositivos; }
    public boolean temAnuncios() { return true; }
    public double calcularMensalidade() { return 0.0; }

    public String resumo() {
        return nome + ": R$ " + calcularMensalidade()
            + " por mes, " + maxDispositivos + " dispositivo(s)";
    }
}
```

```
public class PlanoIndividual {

    private String nome;
    private int maxDispositivos;
    private double precoMensal;

    public PlanoIndividual(double precoMensal) {
        this.nome = "Individual";
        this.maxDispositivos = 1;
        setPrecoMensal(precoMensal);
    }

    public String getNome() { return nome; }
    public int getMaxDispositivos() { return maxDispositivos; }
    public double getPrecoMensal() { return precoMensal; }

    public void setPrecoMensal(double precoMensal) {
        if (precoMensal <= 0) {
            throw new IllegalArgumentException("Preco deve ser positivo");
        }
        this.precoMensal = precoMensal;
    }

    public boolean temAnuncios() { return false; }
    public double calcularMensalidade() { return precoMensal; }

    public String resumo() {
        return nome + ": R$ " + calcularMensalidade()
            + " por mes, " + maxDispositivos + " dispositivo(s)";
    }
}
```

```
public class PlanoFamilia {
```

```
private String nome;
private int maxDispositivos;
private double precoMensal;
private int quantidadeMembros;

public PlanoFamilia(double precoMensal, int quantidadeMembros) {
    this.nome = "Familia";
    this.maxDispositivos = 6;
    setPrecoMensal(precoMensal);
    setQuantidadeMembros(quantidadeMembros);
}

public String getNome() { return nome; }
public int getMaxDispositivos() { return maxDispositivos; }
public double getPrecoMensal() { return precoMensal; }
public int getQuantidadeMembros() { return quantidadeMembros; }

public void setPrecoMensal(double precoMensal) {
    if (precoMensal <= 0) {
        throw new IllegalArgumentException("Preco deve ser positivo");
    }
    this.precoMensal = precoMensal;
}

public void setQuantidadeMembros(int quantidadeMembros) {
    if (quantidadeMembros < 1 || quantidadeMembros > 6) {
        throw new IllegalArgumentException("Membros deve ser de 1 a 6");
    }
    this.quantidadeMembros = quantidadeMembros;
}

public boolean temAnuncios() { return false; }

public double calcularMensalidade() {
    return precoMensal + 4.90 * (quantidadeMembros - 1);
}

public String resumo() {
    return nome + ": R$ " + calcularMensalidade()
        + " por mes, " + maxDispositivos + " dispositivo(s)";
}
}
```

Agora aplique o processo de generalização visto em aula, em duas rodadas.

Rodada 1: o que é comum aos três

1. Localize os membros (atributos e métodos) que aparecem nas três classes com o mesmo significado e o mesmo algoritmo.
2. Crie a superclasse Plano e mova para ela esses membros. As três classes passam a herdar de Plano.
3. Os métodos que existem nas três, mas com algoritmo diferente em cada uma, continuam nas classes originais. Pense bem em quais são eles: essa decisão é o centro do exercício.
4. Cada subclasse inicializa a parte herdada chamando `super(...)` no construtor, como você fez na Fase 05.

Rodada 2: o que é comum a um conjunto menor

A generalização pode ser aplicada de novo, agora olhando só para algumas das subclasses. Compare o que sobrou em `PlanoIndividual` e `PlanoFamilia`: ainda há membros repetidos entre as duas que não fazem sentido no plano gratuito.

1. Crie uma superclasse intermediária `PlanoPago`, que herda de `Plano`, e mova para ela o que é comum aos dois planos pagos (incluindo a validação do preço).
2. `PlanoIndividual` e `PlanoFamilia` passam a herdar de `PlanoPago`. `PlanoGratuito` continua herdando direto de `Plano`.
3. Confira que nenhum membro ficou duplicado em duas classes irmãs e que o comportamento continua o mesmo do código do estagiário.

Para pensar. Depois das duas rodadas, olhe para o que sobrou dentro de `PlanoIndividual`. Se você levasse o cálculo simples da mensalidade para `PlanoPago`, a classe ficaria sem nenhum membro próprio. O que a aula diz sobre uma classe que fica vazia depois da generalização? Escreva a sua resposta em um comentário no topo de `PlanoIndividual`, mas mantenha a hierarquia como pedida acima.

Parte C: classes e métodos abstratos

Repare no efeito colateral da generalização: `Plano` e `PlanoPago` surgiram do processo, mas ninguém assina um "plano" genérico nem um "plano pago" genérico. O mesmo vale para o Conteúdo da Fase 05. São conceitos abstratos.

Nos planos

1. Declare `Plano` e `PlanoPago` como `abstract`.
2. O `resumo()` que subiu para `Plano` chama `calcularMensalidade()`, então esse método precisa existir em `Plano`. Em vez de escrever um corpo vazio ou um `return 0` só para compilar, declare `calcularMensalidade()` como método abstrato em `Plano`. Faça o mesmo com `temAnuncios()`.
3. Implemente os métodos abstratos onde fizer sentido. Observe que `PlanoPago`, por ser abstrata, não é obrigada a implementar todos eles: quem não implementar passa a obrigação adiante para as subclasses concretas.
4. Use `@Override` em toda implementação de método abstrato.

No Conteúdo

1. Declare `Conteudo` como `abstract`.
2. Adicione em `Conteudo` o método abstrato `public abstract String getCreditos();`, que devolve quem assina aquele conteúdo. Em `Musica`, algo como o artista e o álbum; em `Podcast`, algo como o número do episódio e o apresentador.
3. Faça o `reproduzir()` de `Conteudo` usar `getCreditos()` na mensagem, por exemplo: Reproduzindo: Bohemian Rhapsody - Queen (A Night at the Opera).

Comprove no App que as classes abstratas não podem ser instanciadas. Deixe as linhas abaixo comentadas no seu App, cada uma com um comentário dizendo qual erro o compilador mostra e por quê:

```
// Conteudo c = new Conteudo("Generico", 120);  
// Plano p = new Plano("Generico", 1);
```

Parte D: final

Alguns métodos têm um algoritmo que não pode ser alterado por uma subclasse, porque isso deixaria o objeto num estado inconsistente. E algumas classes simplesmente não devem ter filhas.

1. Adicione em Conteudo um contador de reproduções (reproducoes, com getter e sem setter público). O reproduzir() passa a incrementar esse contador antes de imprimir a mensagem. Torne reproduzir() final: se uma subclasse sobrescrevesse o método e esquecesse de incrementar, o contador ficaria errado.
1. Torne resumo() final em Plano. O formato do resumo é o mesmo para qualquer plano; o que muda de um para outro é só o cálculo, e esse já está garantido pelo método abstrato.
2. Torne a classe PlanoGratuito final. Regra de negócio do Sonora: ninguém pode especializar o plano gratuito para, por exemplo, sobrescrever temAnuncios() e tirar os anúncios de graça.
3. Comprove no App deixando comentada uma tentativa de estender PlanoGratuito, com um comentário explicando o erro de compilação.

Parte E: integre no Sonora

1. Todo Usuario passa a ter um Plano. Um usuário recém-criado começa no PlanoGratuito.
2. Crie em Usuario o método assinar(Plano novoPlano), que troca o plano do usuário. Plano nulo é erro: lance IllegalArgumentException.
3. No menu do App, adicione as opções para um usuário trocar de plano (gratuito, individual ou família, pedindo o que for necessário para criar o plano escolhido) e para exibir o resumo do plano atual de um usuário.
4. Na demonstração, reproduza algumas músicas e podcasts mais de uma vez e mostre o contador de reproduções de cada um.
5. Trate todas as entradas inválidas com o que você já sabe de exceções. O menu nunca pode quebrar.